



Machine Learning II

Data Science M.Sc.

Prof. Dr. Steffen Wagner
Berliner Hochschule für Technik

Winter term 2025/26

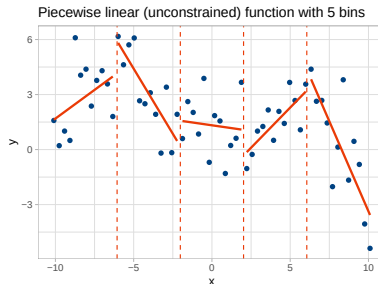


Non-linear regression methods II



Local regression: recap

One of the non-linear function prediction methods we considered last week used piecewise linear regression, where a regression line is fitted to each interval:



- $y_i = f(x_i) = f_0(x_i)C_0(x_i) + f_1(x_i)C_1(x_i) + \dots + f_K(x_i)C_K(x_i) + \epsilon_i$ where each $f_k(x_i) = \beta_{0k} + \beta_{1k}x$ is a linear function.



Local regression: next steps

So far:

- ▶ We quickly moved on to adding constraints such as the piecewise function should be continuous and smooth.

Now we will take this idea in another direction

- ▶ Instead of 5 *disjoint intervals* each with $\frac{1}{5}$ of the points, we will use many *overlapping intervals*.
- ▶ We consider a *window* containing $\frac{1}{5}$ of the data points passing through the range of x .
- ▶ To estimate $f(x_0)$, instead of finding the disjoint interval which contains x_0 , we define a window containing the $\frac{1}{5}n$ points nearest to x_0 .
- ▶ Because we are using $\frac{1}{5}n$ points, the *span* of the window is equal to $\frac{1}{5}$.

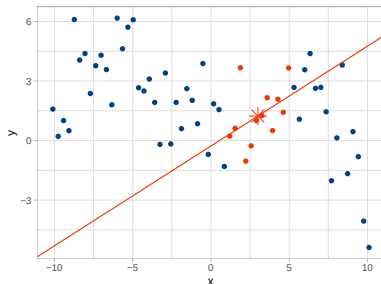


Example

Parameters: $x_0 = 3$ and $n = 60$, so $\lceil \frac{n}{5} \rceil = \lceil \frac{60}{5} \rceil = 12^1$.

Local regression:

using the 12 points nearest to x_0 gives $f(x) = -0.27 + 0.50x$ and $f(x_0) = f(3) = 1.23$.

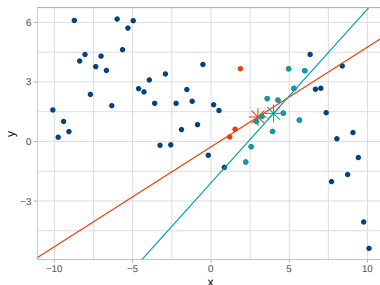


¹non-integers are rounded up.



Example (cont)

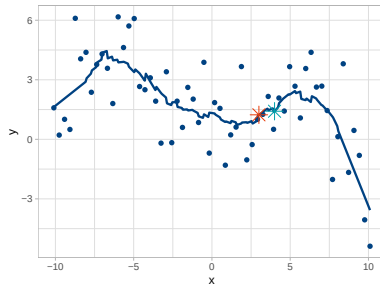
To predict $f(4)$ we use the 12 nearest points to $x_0 = 4$ which includes 9 of the red points on the previous slide and 3 new points.





Example (cont)

We then repeat for all the points on a grid and plot them to give the predictor function $f(x)$.



Note that the method does not require that x_0 is a value in our data, we just need to identify which points are nearest to x_0 .



Loess

The local regression algorithm, called *loess* (locally estimated least squares) or *lowess*, has a couple of important difference to the above explanation.

- ▶ In the above example we computed the regression line giving equal weight to all points in the window.
- ▶ Loess uses weighted regression, so that the points close to x_0 have a large influence on f and points further away, but still within the window, have less influence.
- ▶ As in the introductory example, points outside the window have zero influence.

Weighting:

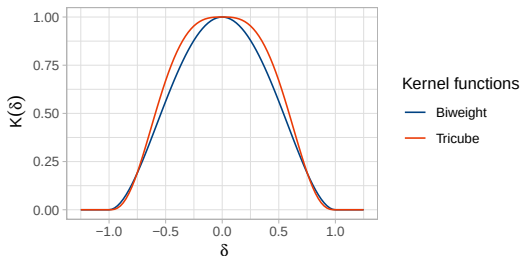
- ▶ The weight function is defined using the distance δ of each x_i from x_0 .
- ▶ The function used to calculate the weights is called a Kernel $K(\delta)$.
- ▶ In loess $\delta \propto x_i - x_0$, so the further away a data point is from x_0 , the smaller $K(\delta)$ is.
- ▶ There are lots of possibilities for the Kernel. They are all symmetric about the y axis, have a maximum at $\delta=0$ and are monotonic decreasing in $|\delta|$.
- ▶ There is a computational advantage when $K(\delta) = 0$ when δ is outside the window.



Kernel functions

The two most common are Kernel functions for loess fitting are

- ▶ Biweight $K(\delta) = (1 - \delta^2)^2$ for $|\delta| \leq 1$ and
- ▶ Tricube $K(\delta) = (1 - |\delta|^3)^3$ for $|\delta| \leq 1$ (the default in R)

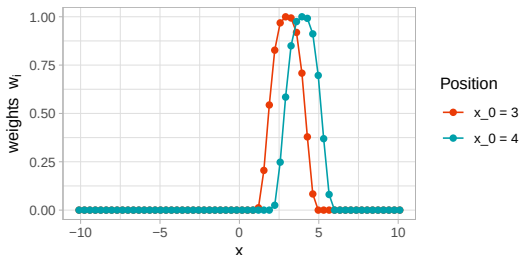


The unweighted example corresponds to using a *uniform* or *boxcar* kernel which is $K(\delta) = 1$ for $|\delta| \leq 1$



Kernel weighting

- ▶ For a given x_0 the weight of the i -th point is $w_i = K\left(\frac{x_i - x_0}{s}\right)$ where s is the distance $s = \max_j |x_j - x_0|$ using the furthest x_j still inside the window.
- ▶ Here are the weights for each data point with a span of $\frac{1}{5}$ and $x_0 = 3$ resp. $x_0 = 4$.



- ▶ As the weight is more localised than when using the uniform kernel it makes sense to use a wider span.



Weighted least squared

In principle the regression is carried out using standard least squares estimation, except that each residual is multiplied by the weight. Minimise

$$RSS = \sum_{i=1}^n w_i (y_i - f_{\beta}(x_i))^2$$

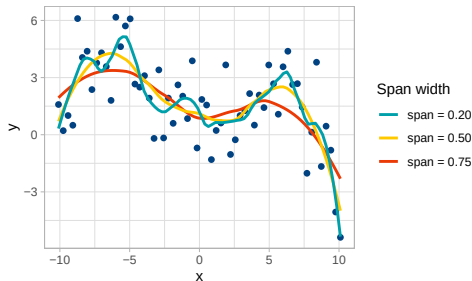
The other difference in the loess implementation is that usually the regression is a locally quadratic regression instead of locally linear regression. The `loess` function in R takes an argument `degree` = to specify locally constant, linear or quadratic.

N.B. If a uniform kernel is used and a constant estimated (`degree=0`) rather than estimating a linear/quadratic regression line, then the method is equivalent to K-Nearest Neighbours regression.



Comments on local regression

- ▶ As with spline smoothing we need to specify the level of smoothing. This is done by defining the span.
- ▶ As the span is usually within the range $[0, 1]$, we have a reasonable starting point for the parameter like specifying the effective degrees of freedom in spline smoothing.

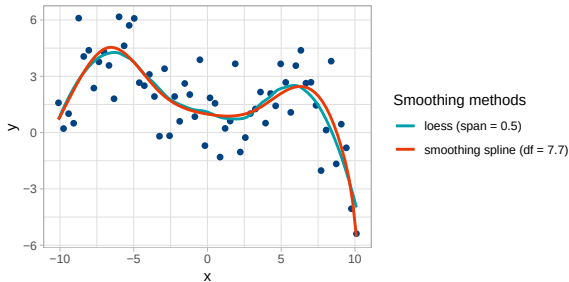


- ▶ The default `span` in R is 0.75.



Loess vs spline smoothing

Loess is a good all-round method which can be used to gain a first impression of the relationship between x and y . So are smoothing splines.



In R some plot methods add a loess curve by default, or make it easy to specify as an option. Example: The diagnostic plots for an R object of class `lm` add a red loess curve as a visual guide to the eye.



Generalised Additive Models (GAM)

All the non-linear regression methods considered so far have used just one predictor variable X . We will now consider how we can adapt these methods to multiple predictor variables $X_1, X_2, \dots, X_p$.

A multiple linear regression model has the form

$$Y_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \dots + \beta_p x_{ip} + \epsilon_i.$$

Non-linear equivalent is the generalised additive Model (GAM)

$$Y_i = \beta_0 + f_1(x_{i1}) + f_2(x_{i2}) + \dots + f_p(x_{ip}) + \epsilon_i.$$

Note: there is another class of statistical models called generalised linear models (GLMs), which include logistic regression. These are not the same!



GAM (cont)

Remarks:

- ▶ There is no constraint that each f_j has to be the same type of function.
So f_1 could be a quadratic function, f_2 a smoothing spline function and f_3 a loess function.
- ▶ As with linear models the predictor variables can be either numeric or factor variables.
- ▶ For convenience, we will assume for the rest of this lecture that each component function is a smoothing spline.
- ▶ The following methods generalise easily to any univariate linear or non-linear regression method.



GAM: model estimation

Q: How do we estimate each of the functions in the GAM?

An iterative approach could be:

1. We can easily estimate $f_1(x_{i1})$ fitting univariate spline smoothing on (x_{i1}, y_i) .
2. To estimate $f_2(x_{i2})$ we need to remove the fitted effect that $f_1(x_{i1})$ has on y_i before using spline smoothing on X_2 .
 $\Rightarrow$ we calculate $\eta_{i2} = y_i - f_1(x_{i1})$, and fit univariate spline smoothing on (x_{i2}, η_{i2}) .
3. To estimate $f_3(x_{i3})$ we need to remove the fitted effect that $f_1(x_{i1})$ and $f_2(x_{i2})$ before using spline smoothing has on y_i before using spline smoothing on X_2 .
 $\Rightarrow$ we calculate $\eta_{i3} = y_i - f_1(x_{i1}) - f_2(x_{i2})$ and fit univariate (x_{i3}, η_{i3}) .

Problem: we didn't consider the impact from $f_2(x_{i2})$ and $f_3(x_{i3})$ on y_i fitting $f_1(x_{i1})$.



Partial residuals

The η values obtained by subtracting out the other components are called *partial residuals*.

The k -th *partial residuals* (for a GAM model) is defined as

$$\eta_{ik} = y_i - \beta_0 - f_1(x_{i1}) - \dots - f_{k-1}(x_{i, k-1}) - f_{k+1}(x_{i, k+1}) - \dots - f_p(x_{ip})$$

for $i = 1, \dots, n$.

Remarks:

- ▶ Notice all the other fitted values are removed **except** $f_k(x_{ik})$
- ▶ The full residuals are the error terms when the full model is fitted.
- ▶ The k -th *partial residuals* is the error term when everything except for f_k is fitted.



Backfitting algorithm for GAMs

The method for obtaining all p estimating functions f_k is called *backfitting*. An insight to the method is below, but the technical details are omitted.

1. Set the constant parameter $\hat{\beta}_0 = \bar{y}$ this remains fixed throughout the algorithm.
2. Initialise all the component functions to $f_k(x) = 0$
3. Iterate over $k \in \{1, \dots, p\}$
 - i) Obtain the k -th partial residuals η_{ik} .
 - ii) Obtain the spline smoothing function $f_k(x)$, which fits the η_{ik} partial residuals.
4. Repeat Step 3 until convergence, i.e. the change in the fitted values for each the component $f_k(x)$ is less than a specified tolerance.



example: PISA scores 2006

The data is constructed using average Science scores by country from the Programme for International Student Assessment (PISA) 2006, along with GNI per capita (Purchasing Power Parity, 2005 dollars), Educational Index, Health Index, and Human Development Index from UN data. The key variables are as follows:

- ▶ **Overall:** Overall Science Score (average score for 15 year olds, based)
- ▶ **Interest:** Interest in science
- ▶ **Issues:** Identifying scientific Issues
- ▶ **Explain:** Explaining phenomena scientifically
- ▶ **Support:** Support for scientific inquiry
- ▶ **Income:** Income Index
- ▶ **Health:** Health Index
- ▶ **Edu:** Education Index
- ▶ **HDI:** Human Development Index (composed of the Income index, Health Index, and Education Index)

source:

<https://m-clark.github.io/generalized-additive-models/application.html>



PISA scores: univariate smoothing



Note the negative trend between Support and Overall!

Suprising?!



PISA scores: GAM modeling

```
library(mgcv)  # THE(!) GAM model package in R
```

```
pisa <- read.csv("https://raw.githubusercontent.com/m-clark/generalized-additive-models/master/data/pisasci2006.csv")
```

```
gam_01 <- gam(Overall ~ s(Support, bs = "cs")  
              + s(Income, bs = "cs")  
              + s(Health, bs = "cs")  
              + s(Edu, bs = "cs"),  
              data = pisa)
```

```
# s() ist the spline function in the 'mgcv' package. The type of spline you want to use  
# is specified via the argument `bs = ...`  
# check `?smooth.terms` to get an overview about the zoo of spline methods
```

```
# summary method: output next slide  
summary(gam_01)
```



PISA scores: GAM modeling

Family: gaussian

Link function: identity

Formula:

```
Overall ~ s(Support, bs = "cs") + s(Income, bs = "cs") + s(Health,
  bs = "cs") + s(Edu, bs = "cs")
```

Parametric coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	471.154	2.735	172.3	<2e-16 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Approximate significance of smooth terms:

	edf	Ref.df	F	p-value
s(Support)	0.000005205	9	0.000	0.54688
s(Income)	6.893267429	9	8.842	< 2e-16 ***
s(Health)	0.592709205	9	0.142	0.12612
s(Edu)	6.081791479	9	2.887	0.00121 **

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

R-sq.(adj) = 0.867 Deviance explained = 90.2%

GCV = 540.18 Scale est. = 388.85 n = 52



PISA scores: model refinement

Checking the model summary given on the previous slide, we see that ...

- ▶ the variable `Support` is not significant, when the variables are considered simultaneously.

So we remove this variable from the model equation.

- ▶ the effective degrees of freedom (`edf`) for `Health` is ≤ 1 , which means we could model this variable with a simple linear term. Since `Health` is also not significant we remove it as well.

So we update the model as follows:

```
gam_02 <- gam(Overall ~ s(Edu, bs = "cs") + s(Income, bs = "cs"), data = pisa)
```



PISA scores: model refinement

Family: gaussian

Link function: identity

Formula:

Overall ~ s(Edu, bs = "cs") + s(Income, bs = "cs")

Parametric coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	471.154	2.784	169.2	<2e-16 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Approximate significance of smooth terms:

	edf	Ref.df	F	p-value
s(Edu)	5.933	9	2.707	0.00172 **
s(Income)	6.958	9	10.449	< 2e-16 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

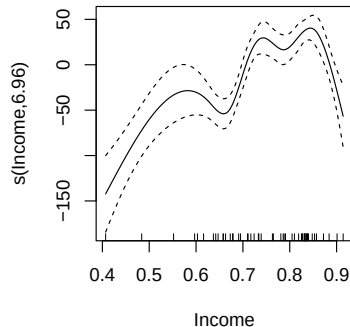
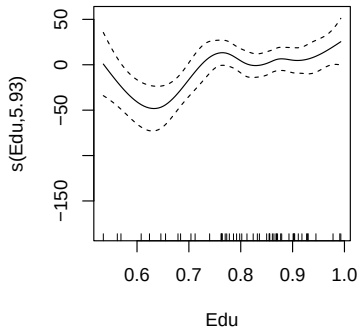
R-sq.(adj) = 0.862 Deviance explained = 89.7%

GCV = 549.89 Scale est. = 402.99 n = 52



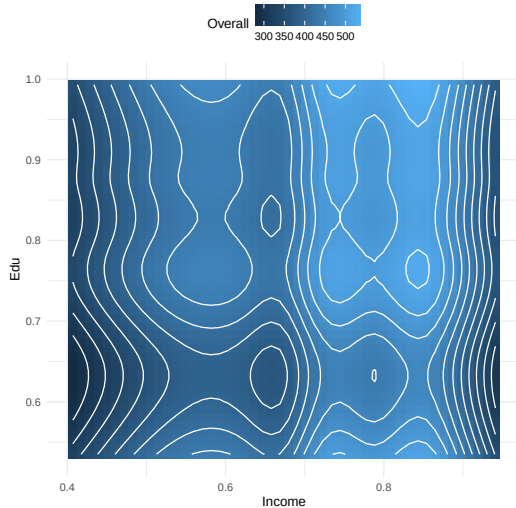
PISA scores: estimated splines

```
plot(gam_02, pages = 1, scale = -1)
```





PISA scores: 2-dim visualisation of estimated splines





GAM: comments

- ▶ GAMs allow us to fit a non-linear predictor to each variable.
- ▶ We will find non-linear relationships that standard multiple linear regression will miss.
- ▶ We do not need to manually try out many different transformations on each variable individually.
⇒ the data *makes* the functional relationship
- ▶ The non-linear fits can potentially make more accurate predictions for the response Y .
- ▶ Because the model is additive, we can examine the effect of each X_j on Y individually while holding all of the other variables fixed.
- ▶ The smoothness of the function f_j for the variable X_j can be controlled independently for each variable.



GAM and categorical features

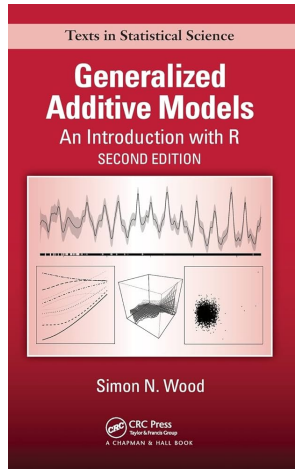
Up to now we have assumed all of the variables are continuous, or at least numeric. Factor variables are usually modelled using one parameter for each factor level.

- ▶ These can be directly added to the GAM with no need for fitting a function.
- ▶ Eg. In the GPA data, if there were another variable for type of high school: public vs private.
- ▶ A GAM is restricted to be additive. Important interactions might be missed.
- ▶ **But:** We can manually add an interaction term to the GAM model by adding a predictor for $X_j \cdot X_k$.

This is normally only done if an interaction term between two variables is expected, because checking for all combinations of interactions adds a large computational overhead.



Further reading



Accompanying R package: `mgcv`